Document number:	P4220R0
Date:	2026-05-08
Audience:	LEWG
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Design goals for `zstring_view`

During the 2025 Sofia meeting, LEWG declared consensus to spend more time on `zstring_view` ([P3655R4]). This paper follows the direction. We want to make sure that the design goals for `zstring_view` are clearly understood before the decision whether to even have it or not are made.

Terminology

Contract

In this paper, whenever the word 'contract' is used, we *do not* refer to the C++26 feature known as "contract assertions", but instead refer to the methods of Library API specifications.

C-string contract

The term *C-string contract* refers to the convention that the C programming language uses for representing strings:

1. The type is `const char*`
2. The pointer is not null.
3. The pointer points to a valid array of characters.
4. The iteration over the array is guaranteed to reach the zero character without hitting any undefined behavior. The position of this zero character determines the string size.
5. A zero-size string is a valid string value.

Note that under this contract it is impossible for the string to have a zero character within its contents, as — per the contract — the first occurrence of this character indicates the one-past-last character.

Note also that in the C library there are functions that take

- (`const char*`, `size_t`), like `strnlen`, where the size of the string is determined as the smaller of "the distance from the closest zero character" and the explicit length;
- (`const void*`, `size_t`), like `fwrite`, which can in particular work for arrays of `char` and where the length is provided explicitly and zero characters can appear in the middle.

They are excluded from the definition of the *C-string contract*.

The motivation

A proposal should set out with a very clear goal statement, so that LEWG can evaluate:

- if the goal is worth pursuing;
- if the proposed solution addresses the stated goal optimally.

"A lot of people ask for it" or "there is a lot of similar GitHub libraries" or "`std` does not yet have a type with this combination of properties" is not a sufficient motivation.

A lot of people ask that C++ provides a `finally` keyword. The correct response in that case has been to educate the people on C++'s destructors rather than just fulfilling the request.

Similarly, a lot of people may demand "a type like `cstring_view`", but each of them may mean slightly different incompatible semantics, addressing slightly different incompatible goals.

The motivation that we have identified in [P3655R4] is the experience where:

1. The author of the function will be calling a system function, such as POSIX open.
2. The author insists on using `std::string_view` as the function parameter type rather than `const char*`.

Such setup "compiles" and could work, but the callers would need to be informed and then disciplined to only create `string_view` objects that happen to be zero-terminated. This is called a *precondition*. Other options that the author has are to either use `const char*` as the function parameter type along with the C-string contract, or introduce a new type that directly reflects the C-string contract. In all three cases, the fact that a pointer to a zero-terminated character array will be used in the implementation is exposed in the function's contract, either as a precondition or as a dedicated type (because if it weren't, we would have just used `string_view`).

So, given that the C-string contract will be used anyway, why not just use `const char*` as the function parameter type, rather than insisting on a new type? The answers could be:

1. To reflect in the type the C-string contract: pointer must not be null, the array shall contain the zero character. Even if this is just `const char*`, let the people see the unusual new type name.
2. To enable the type, if so configured, to runtime-enforce the C-string contract.
3. To avoid the misleading semantics of `operator==` for `const char*` with the C-string contract.
4. To offer a type with the C-string contract which additionally keeps the length explicitly, which may be obtained in $\mathcal{O}(1)$ time for some constructors.

Let's illustrate the last bullet. This would be in the situation where a program receives a zero-terminated string from one C-style API, then plays with it using the `string_view` interface, and finally passes it to another C-style API that uses the C-string contract:

```
void demonstration()
{
  const char * s = clib1::get_str();     // #1
  play1(string_view(s).starts_with("pre"));
  play2(string_view(s).find_last_not_of('_'));
  clib2::use_str(s);
}
```

If the string size could be computed in line #1, then in lines #2 and #3 we could use it for free.

Ultimately, however, the `const char*` will be passed to the system function open which will not care whether we know the string size or not: it will unconditionally iterate over the array until the zero terminator anyway. No design in `zstring_view` can change that.

In the following analysis we often assume that the goal of the new type is to be a "C-string contract enforcer". [P3655R4], doesn't state its goal clearly enough.

Design decisions

Constructors

The C-string semantic contract consists of three parts.

1. Checking if the pointer is non-null is trivial.
2. Checking if the pointer points to a valid character array is impossible.
3. Checking for a zero character is tricky: we could iterate over the elements to look for it, but if it isn't there we will reach UB.

If the primary goal of `zstring_view` is to runtime-enforce the C-string contract, then the whole point is to be able to test #3 above. This is doable if in the constructors we are additionally provided the limit for the iteration. It is easy to do for some constructors:

1. `zstring_view(string const& s)` — `s` already guarantees the null terminator.
2. `zstring_view(const char(&a)[N])` — `N` (a template parameter) is the limit, and we can check for zero at `N - 1`.
3. Copy/move constructor — it is safe to assume that we already performed the check when creating the original.

But we definitely cannot accept `zstring_view(const char*)`, as proposed in [P3655R4], because we will not be able to verify the contract. So either we explicitly abandon the goal "C-string contract enforcer" or consider an alternative design where the conversion from `const char*` is removed. This means that the simplest and intuitive use cases will not work:

```
catch(std::exception const& exc)
{
  string s = exc.what();        // ok
  string_view sv = exc.what();  // ok
  zstring_view zv = exc.what(); // compiler error!
}
```

If we allow a constructor from `const char*` we have failed to achieve the "runtime-enforce the C-string contract" goal.

If we only provide constructor `zstring_view(const char*, size_t)`, we either make the usage of this type impossible, or bug prone:

```
catch(std::exception const& exc)
{
  size_t Max = 128;  // arbitrary size limit
  zstring_view zv(exc.what(), Max); // compiles, but may be UB
}
```

The goal "C-string contract enforcement", if it is the goal, seems unimplementable.

We could consider a slightly different goal instead, and say that the new type *either* allows a runtime-verifiable correct construction *or* provides a very explicit syntax for uncheckable initialization that is easy to audit:

```
catch (std::exception const& exc)
{
  auto zv = zstring_view::RISKY_convert(exc.what()); // ok
}
```

Such `zstring_view` would still be far from being a drop-in replacement for `const char*` in function parameters.

Depending on what the goal of `zstring_view` is, a different set of constructors may be optimal. But we will not be able to assess which constructor set is optimal, until we know the design goal.

The richness of the interface

If the goal is to have a C-string interface with the ability to runtime-enforce it, then the excessive richness of the `std::string` interface (such as function `find_last_not_of`) is not necessary.

Compare the different contracts of `string`, `string_view` and `zstring_view`.

1. `string` — a `char` container that additionally exposes the string-rich interface. Zero is a perfectly valid element value.

2. `string_view` — an arbitrary sub-sequence of another `char` sequence managed elsewhere, which also exposes string-rich interface. Zero is a perfectly valid element value.
3. `zstring_view` — a `const char*` which can additionally runtime-enforce the C-string contract. By contract definition, it cannot have zero characters in the middle, and it will only be passed to function open.

The only interface of `zstring_view` that will be used in practice is its constructors and function `.data()`. Even `operator[]` is unnecessary: just call `.data()` and iterate over this array.

We lose the string-rich interface, but do we need it? If so, we can convert to `string_view`. This would be an $\mathcal{O}(n)$ cost if the goal is "C-string contract enforcer", or an $\mathcal{O}(1)$ cost if the goal is "C-string contract + precomputed size". In the latter case we would penalize the most basic use case:

```
zstring_view zs = get_c_string(); // 𝒪(n) pass
open(zs.data());          // another 𝒪(n) pass
```

Thus, the decision whether to provide the rich or the minimum interface hinges on selecting the design goal first.

String's length

`string` stores its length explicitly, so that it can treat the zero-character as an ordinary character. `string_view` stores its length because this is necessary to represent a subsection of a longer character sequence, and because zero is a valid element of that sequence.

In contrast, the contract of `zstring_view` is that it will ultimately be passed to a function like POSIX open and its size will be determined by iterating throughout the sequence until the zero-character. The iteration will be performed, no matter what we do! Keeping the precomputed length doesn't add value here, but on the other hand would pose a new problem: this explicit size and the result of `strlen` would have to be kept in sync, and this is difficult when zero characters are present in the sequence before the end of explicit length.

```
string s("A\0B", 3);
string_view sv("A\0B", 3);
zstring_view zs("A\0B", 3);

assert (s.length() != strlen(s.data()));   // ok: `strlen(s.data())` is not the length
assert (sv.length() != strlen(sv.data())); // ok: `strlen(sv.data())` is not the length
assert (zs.length() != strlen(zs.data())); // disaster: `strlen(zs.data())` is the length
```

Assuming the goal of `zstring_view` is "a `const char*` that additionally verifies the C-string contract", consistent solutions would be:

1. Do not provide member `length()` or `size()`.
2. Make these members equivalent to `strlen(data())`.
3. Add a precondition, potentially runtime-enforced, that zero-characters in the middle are not allowed.

It may still make sense to have the `size_t` member but with a different interpretation. If `zstring_view` provides `operator[]`, this member could be used as an aid to runtime-enforce the precondition that the index is "in the right range".

Option #2 still requires other questions to be answered: shall this value of length be computed upon construction? If so, this a waste in the very basic use case:

```
zstring_view zs("may contain a \0 char"); // 1st range iteration
fopen(zs.data());                         // 2nd range iteration
```

We could compute the length when it is first needed and then cache the result. But this causes data-race issues.

What do current implementations do?

Different implementations do different things and pursue — consciously or not — different design goals. In fact, whoever decides to implement "something like `zstring_view`" is not obliged to state or follow any design goals. Therefore, there is a limit to how much such research can help guide the design for a Standard Library component, where the design bar is higher: it should be founded on principles. But we can explore some implementations.

Beman Project » `cstring_view` (https://github.com/bemanproject/cstring_view)

The library offers a conversion from `const char*` with UB if the `char` array is not zero-terminated. Length is eagerly computed in the constructor, and can be later retrieved in $\mathcal{O}(1)$. Middle-zeros are allowed.

However, libraries from Beman Project are not a good fit for studying the design. They are meant to be a proof of implementability for already proposed libraries (where design goals had been stated). The `cstring_view` library is documented as implementing [P3655R2], so it cannot be used to inspire its design. That would be circular.

Microsoft's GSL » `zstring` (https://github.com/microsoft/GSL/blob/main/include/gsl/zstring)

Microsoft's GSL used to have more types dedicated to enforcing the C-string contract, but since version 4.0.0, they become obsolete ([GSL400]) and the only thing that is left is `zstring`. It is a type alias on `char*`. No runtime enforcements, just a name marker. This is what C++ Core Guidelines ([CPPGUIDE]) end up recommending using.

NVIDIA's implementation

We do not have access to NVIDIA's implementation of `cstring_view`, but we can gather from the description in [P3655R4] that the likely goal was to enable a gradual modification of the code base. If so, it required an $\mathcal{O}(1)$ conversion from `cstring_view` to `string_view`. This appears close to "A C-string with additional precomputed length".

Implementations of "something like `zstring_view`" exist in quantity, but they do not necessarily agree on their primary goal, often they state no goal.

Recommendations

LEWG should not approve any paper proposing a library component that does not clearly state its design goal. This is necessary for everyone in WG21 to be clear on what the goal is, to be able to assess if the goal is worth pursuing, and if the proposed solution actually addresses the goal.

We observe that [P3655R4] does not express the goal clearly enough. Without this LEWG cannot design the type properly. It can only poll who likes which function better.

The observation that many people demand to have a type called "zstring_view" and that many people implemented their type called "zstring_view" is misleading. These implementations by different parties have different semantics and serve different goals. As we have shown, designing for one goal compromises other possible goals.

If the goal for `std::zstring_view` is not clearly stated the worst projected outcomes may be:

1. We end up with an uncoordinated combination of design choices that in total satisfy nobody.
2. Experts caught off guard when making decisions on the paper may unconsciously mistake their assumptions on what a "zstring_view" represents for what is actually proposed.

While [P3749R0] raises other objections against [P3655R4] our paper focuses solely on defining the goal clearly. Once this is settled, only then can we start a due critique based on the stated goal, including the reevaluation of [P3749R0].

References

- [CPPGUIDE] — Bjarne Stroustrup, Herb Sutter, "C++ Core Guidelines", ("https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#gslview-views").
- [GSL400] — Dmitry Kobets, "GSL 4.0.0 is Available Now", ("https://devblogs.microsoft.com/cppblog/gsl-4-0-0-is-available-now/").
- [N3442] — Jeffrey Yasskin, "string_ref: a non-owning reference to a string", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3442.html").
- [P2176R0] — Andrzej Krzemieński, "A different take on inexpressible conditions", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2176r0.html").
- [P3081R1] — Herb Sutter, "Core safety profiles for C++26", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3081r1.pdf").
- [P3655R2] — Peter Bindels, Hana Dusíková, Jeremy Rifkin, Marco Foco, Alexey Shevlyakov, "std::zstring_view", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3655r2.html").
- [P3655R4] — Peter Bindels, Hana Dusíková, Jeremy Rifkin, Marco Foco, Alexey Shevlyakov, "std::cstring_view", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3655r4.html").
- [P3697R0] — Konstantin Varlamov, Louis Dionne, Alisdair Meredith, "Minor additions to C++26 standard library hardening", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html").
- [P3705R2] — Eddie Nolan, "A Sentinel for Null-Terminated Strings", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3705r2.html").
- [P3749R0] — Jan Schultke, "Slides in response to P3655R2 Concerns regarding std::zstring_view", ("https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3749r0.html").